

We assessed commit [#6f6486e3a0df149690487316e20ca3d1e45f5cc7](#)

Notes for you/your team

Behavior

Misc notes:

- What does it do? (business purpose)
- This has a chatbot feature to it, so it def is using AI. Are there any agentic parts of this application.

Projects have tasks, tasks have notes, and users are assigned based off of some numerice value to *projects*

- Who does it do this for? (internal / external customer base)

I noticed the fixture data has users with both tm and outsider email addresses - does that mean we're comingling data between the company creating the product and the consumers using it.

- What kind of information will it hold?

DOB and SSN for sure

- What are the different types of roles?

[admins, team members, and project managers](#)

There are two independent, overlapping authz mechanisms in play:

1. **Groups/permissions (RBAC)** - Users belong to `auth.Groups` (`taskManager/fixtures/groups.json`), and permissions are assigned to those groups, not to individual users (`taskManager/fixtures/auth_group_permissions.json`). Django's `user.has_perm(...)` resolves permissions transitively through group membership, which is why every user's `user_permissions` field (direct, per-user grants) is empty - it's simply unused; all perm checks (`can_create_project`, `can_edit_project`, `can_delete_project`, `manage_groups`'s `auth.change_group` check, etc. in `taskManager/views.py`) rely on group-derived permissions instead.
2. **`is_staff` / `is_superuser` flags** - These are separate boolean fields on the `User` model, unrelated to the group/permission system. `is_superuser` implicitly grants every permission (Django built-in behavior) and is set on `admin/seth` in the fixtures. `is_staff` is used as a custom "trusted/internal" flag throughout the codebase - e.g. `UserViewSet`, `TaskViewSet`, `ProjectViewSet`, `UserProfileViewSet` in `taskManager/serializers.py` and `_project_accessible/_task_accessible/_note_accessible` in `chatbot/tools.py` all bypass normal ownership/membership filtering and return **all** records when `user.is_staff` is true. It is not tied to Django admin access here - it's being (re)purposed as a broad "can see everything" authz bit, layered on top of the group/permission RBAC system.

Net effect: a user's access is the union of whatever their group permissions allow *and* whatever `is_staff/is_superuser` unlocks - two authz paths to audit, not one.

- What aspects concern your client/customer/staff the most?

Tech Stack

- Framework & Language
 - Django
 - Python 3
- 3rd party components, Examples:
- Integrated with OpenAI

Library	Functionality	Security Risk (1–5)	Why
Django==5.1.4	Web framework	5	Core application framework. Authentication, sessions, routing, templates, ORM, middleware, CSRF, etc. Compromise or misconfiguration affects the entire application.
djangorestframework	REST API framework	5	Exposes API endpoints, serialization, authentication, permissions, and request handling. Directly affects attack surface.
djangorestframework-simplejwt	JWT Authentication	5	Handles authentication tokens. Misconfiguration can lead to authentication bypass or token abuse.
redis	Caching / Data Store	4	Stores application data, sessions, or cache. Exposure or insecure configuration can impact confidentiality and availability.
openai	LLM API Client	4	Sends prompts and potentially sensitive data to external AI services. Requires API key protection and data handling controls.
requests	HTTP Client	4	Makes outbound network requests. Can introduce SSRF or data exfiltration risks if user-controlled URLs are fetched.

Library	Functionality	Security Risk (1–5)	Why
<code>drf-spectacular</code>	OpenAPI / API Documentation	3	Generates API documentation. Can unintentionally expose API structure if publicly accessible.
<code>django-filter</code>	API Filtering	3	Enables query filtering. Poor configuration can expose unintended data or enable expensive queries.
<code>django-health-check</code>	Health Monitoring	3	Exposes application health endpoints. Should not leak sensitive operational information.
<code>python-decouple</code>	Configuration Management	3	Loads secrets and configuration from environment variables. Incorrect use can expose credentials.
<code>psutil</code>	System Information	3	Provides process and system metrics. May expose sensitive host information if surfaced via endpoints.
<code>asgiref</code>	ASGI Support	2	Django asynchronous interface library. Infrastructure component with limited direct security exposure.
<code>sqlparse</code>	SQL Parsing	2	SQL formatting/parsing utility used internally by Django. Limited direct security impact.
<code>pytz</code>	Time Zone Support	1	Date/time utilities. Minimal security relevance.
<code>xlwt</code>	Excel (.xls) Generation	1	Generates spreadsheet files. Limited security impact unless exporting sensitive data.
<code>django-extensions</code>	Development Utilities	1	Developer tooling (shells, commands, graphing, etc.). Typically not used in production request handling.

Library	Functionality	Security Risk (1–5)	Why
<code>pipenv</code>	Dependency / Virtual Environment Management	1	Development and packaging tool. Not part of the runtime application attack surface.
• Datastore ◦ MYSQL			

Brainstorming / Risks

- I noticed there is an OpenAI API key being used, is it insecurely stored/used/etc.
- I noticed text and title for projects/tasks/notes are a place of input validation
- Appears that users can use md5 for passwords. Does that mean and we need to investigate (however you do that in this app) what password hashing their using.
- The last login seems to be null. Is that an issue? Does the db allow that? Does the functionality even work?
- We saw the user profile has images that seem to be stored on a filepath so if we have file handling, image resizing/etc we may have more risks to investigate around file uploads/downloading/handling.
- Investigate xlwt so we know if we have csv injection, and if the excel file contains macros, and how far with exploitation that could bring us.
- Investigate prompt injection and prevention since this is a chatbot as well as what data sources is it connected and how well are they ensuring data streams don't cross
- Did not see any authorization testing matrix or many robust security tests so we really need to investigate authorization/rbac

Checklist of things to review

Risks

Authentication

Authorization

- ☒ Identify Roles
 - Groups (RBAC via `auth_group_permissions.json`): admins, team members, project managers - permissions attached to groups, not users (`user_permissions` is always empty).
 - `is_superuser` - Django built-in, implicitly grants all permissions (`admin`, `seth` in fixtures).

- `is_staff` - repurposed as a custom "trusted/internal" bit that bypasses row-level filtering in `taskManager/serializers.py` DRF viewsets and `chatbot/tools.py` accessibility checks; not the same thing as `is_superuser`.
- ☐ Identify sensitive/privileged endpoints
 - `taskManager:profile_by_id (/taskManager/profile/<user_id>)` - edits **any** user's profile, group membership, and password by ID.
 - `taskManager:manage_groups, taskManager:view_all_users` - group/user administration.
 - `taskManager:project_create / project_edit / project_delete` - gated by `has_perm('taskManager.<action>_project')`, confirm group->permission mapping matches intended role split.
 - `taskManager:upload, download, download_profile_pic` - file read/write by ID/path.
 - `taskManager:ping, taskManager:tm_settings, taskManager:view_img` - no `@login_required` at all.
 - `chatbot:*` - tool-driven read (and write) access to projects/tasks/notes/users on behalf of the requesting user.
 - DRF `/api/*` viewsets - separate `TokenAuthentication` + `is_staff`-scoped querysets.
- ☐ Identify authz expectations specific to the business purpose of the app
 - Can non-privileged users view, add, or alter accounts? - **Yes:** `profile_by_id` lets any authenticated user load/edit another user's profile (incl. DOB/SSN/password) purely by supplying their numeric `user_id`, with no ownership or permission check.
 - Is there functionality to add accounts with higher access levels than their own access? - **Yes:** `profile_by_id`'s POST handler reads `groups` from the request body and calls `user.groups.add(...)` with no validation the requester is allowed to grant that group - a low-privilege user can add themselves (or anyone) to an admin group (mass assignment -> privilege escalation).
 - How is separation of duties handled? - Solely through group->permission assignments in fixtures (`can_create_project/can_edit_project/can_delete_project` are independent perms); verify no single default group holds all three, and that `manage_groups`'s `auth.change_group` check isn't itself grantable via the `profile_by_id` group-add bug above.
- ☐ Identify Authorization functions/filters
 - `@login_required / @user_passes_test(...)` (Django decorators) - operate on `request.user`, which is populated by custom `taskManager/middleware.py::JWTAuthenticationMiddleware` from a `access_token` **cookie** (JWT, HS256, signed with the hardcoded weak `SECRET_KEY`), not Django sessions.
 - DRF API uses a **separate** credential: `TokenAuthentication` header (`Authorization: Token <key>`) + `IsAuthenticated/is_staff`-based `get_queryset` scoping - confirm the two auth paths can't be mixed/confused (e.g. JWT cookie granting API token-level access or vice versa).
 - `has_perm(...)` resolution is entirely group-derived; confirm no path lets a user's `user_permissions` be silently populated and diverge from group state.
- Broken Access Control
 - ☐ Insecure Direct Object Reference (`find_by, find, findOne, findAll`, etc)

- `profile_by_id/profile_view` (arbitrary `user_id`),
`download/download_profile_pic` (arbitrary `file_id/user_id`),
`task_details/task_edit/task_delete/note_*` (project/task ownership only partially checked - see TODOs below).
 - ☐ Missing Function Level Access Control
 - `task_delete` has an explicit `# TODO: Check authorization` comment; `task_complete` has `# TODO: additional task completion checks`.
 - `ping`, `tm_settings`, `view_img` have no `@login_required` at all - unauthenticated access to command exec, `request.META` dump, and arbitrary URL rendering respectively.
 - ☐ Verify Authorization Filters
 - Confirm `is_staff`-gated `get_queryset()` overrides in `UserViewSet/TaskViewSet/ProjectViewSet/UserProfileViewSet` and `chatbot/tools.py`'s `_project_accessible/_task_accessible/_note_accessible` are intentional and don't leak data to unintended "staff" users.
- Generic authz flaws
 - ☐ Sensitive Data Exposure
 - DOB/SSN readable (and writable) via `profile_by_id` for any target user; `tm_settings` renders raw `request.META`; chatbot `search_database/get_users` tools expose broad user/profile/note text fields.
 - ☐ Mass Assignment
 - `profile_by_id` accepts a raw `groups` field from `request.POST` and adds the target user to any named group with no allow-list or permission check - direct privilege-escalation vector.
 - ☐ Business Logic Flaws
 - `forgot_password/reset_password` use raw SQL keyed on user-supplied email (SQLi); Redis-based lockout in `login` keyed on username only, no CAPTCHA/backoff review.
 - ☐ Are CSRF Protections applied correctly
 - CSRF middleware is globally disabled (`settings.py MIDDLEWARE`), and several state-changing views also carry an explicit `@csrf_exempt` (`profile_by_id`, `reset_password`, `forgot_password`, `change_password`, `ping`) - double-check none of these were meant to be re-protected.
 - ☐ Are users forced to re-assert their credentials for requests that have critical side-effects (account changes, password reset, etc)?
 - `change_password` sets a new password from `new_password/confirm_password` with no current-password check; `profile_by_id` can set `password` for another user entirely with no re-auth.

Auditing/Logging

- ☐ Logging configuration is in `settings.py`, check documentation for secure settings

Injection

- ☐ ORM **where** function allows for string concatenation, search for all instances

Cryptography

- ☐ References to base64 when handling passwords, is this bad?

Configuration

- ☐ Code is ruby/rails, make sure and run brakeman before closing out

Mapping / Routes

Prefix URI Pattern Controller#Action

High

- ☐ rest_framework:login /api-auth/login/ django.contrib.auth.views.view
- ☐ rest_framework:logout /api-auth/logout/ django.contrib.auth.views.view
- ☐ /api-token/ rest_framework.authtoken.views.view
- ☐ taskManager:login /taskManager/login/ taskManager.views.login
- ☐ taskManager:logout /taskManager/logout/ taskManager.views.logout_view
- ☐ taskManager:register /taskManager/register/ taskManager.views.register
- ☐ taskManager:forgot_password /taskManager/forgot_password/
taskManager.views.forgot_password
- ☐ taskManager:reset_password /taskManager/reset_password/
taskManager.views.reset_password
- ☐ taskManager:change_password /taskManager/change_password/
taskManager.views.change_password
- ☐ taskManager:profile /taskManager/profile/ taskManager.views.profile
- ☐ taskManager:profile_by_id /taskManager/profile/<user_id> taskManager.views.profile_by_id
- ☐ taskManager:profile_view /taskManager/profile_view/<user_id> taskManager.views.profile_view
- ☐ taskManager:manage_groups /taskManager/manage_groups/
taskManager.views.manage_groups
- ☐ taskManager:view_all_users /taskManager/view_all_users/ taskManager.views.view_all_users
- ☐ taskManager:ping /taskManager/ping/ taskManager.views.ping
- ☐ file-list /api/files/ taskManager.serializers.FileViewSet
- ☐ file-detail /api/files/ taskManager.serializers.FileViewSet
- ☐ file-detail /api/files/. taskManager.serializers.FileViewSet
- ☐ file-list /api/files/ taskManager.serializers.FileViewSet
- ☐ taskManager:upload /taskManager/<project_id>/upload/ taskManager.views.upload
- ☐ taskManager:download /taskManager/download/<file_id>/ taskManager.views.download
- ☐ taskManager:download_profile_pic /taskManager/downloadprofilepic/<user_id>/
taskManager.views.download_profile_pic
- ☐ taskManager:view_img /taskManager/view_img/ taskManager.views.view_img
- ☐ /uploads/ django.views.static.serve
- ☐ chatbot:chat_page /chat/page/ chatbot.views.chat_page
- ☐ chatbot:chat_send /chat/send/ chatbot.views.chat_send
- ☐ chatbot:session_delete /chat/session/delete/ chatbot.views.session_delete

- ☐ chatbot:session_messages /chat/session//messages/ chatbot.views.session_messages
- ☐ chatbot:session_new /chat/session/new/ chatbot.views.session_new
- ☐ chatbot:session_list /chat/sessions/ chatbot.views.session_list
- ☐ chatbot:chat_stream /chat/stream/ chatbot.views.chat_stream
- ☐ health_check:health_check_home /ht/ health_check.views.view
- ☐ health_check:health_check_subset /ht/[str:subset](#)/ health_check.views.view
- ☐ taskManager:search /taskManager/search/ taskManager.views.search

Medium

- ☐ notes-list /api/notes/ taskManager.serializers.NotesViewSet
- ☐ notes-detail /api/notes// taskManager.serializers.NotesViewSet
- ☐ notes-detail /api/notes./ taskManager.serializers.NotesViewSet
- ☐ notes-list /api/notes./ taskManager.serializers.NotesViewSet
- ☐ project-list /api/projects/ taskManager.serializers.ProjectViewSet
- ☐ project-detail /api/projects// taskManager.serializers.ProjectViewSet
- ☐ project-detail /api/projects./ taskManager.serializers.ProjectViewSet
- ☐ project-list /api/projects./ taskManager.serializers.ProjectViewSet
- ☐ task-list /api/tasks/ taskManager.serializers.TaskViewSet
- ☐ task-detail /api/tasks// taskManager.serializers.TaskViewSet
- ☐ task-detail /api/tasks./ taskManager.serializers.TaskViewSet
- ☐ task-list /api/tasks./ taskManager.serializers.TaskViewSet
- ☐ taskManager:task_details /taskManager/<project_id>/<task_id>/
taskManager.views.task_details
- ☐ taskManager:note_create /taskManager/<project_id>/<task_id>/note_create/
taskManager.views.note_create
- ☐ taskManager:note_delete /taskManager/<project_id>/<task_id>/note_delete/<note_id>
taskManager.views.note_delete
- ☐ taskManager:note_edit /taskManager/<project_id>/<task_id>/note_edit/<note_id>
taskManager.views.note_edit
- ☐ taskManager:project_edit /taskManager/<project_id>/edit_project/
taskManager.views.project_edit
- ☐ taskManager:manage_tasks /taskManager/<project_id>/manage_tasks/
taskManager.views.manage_tasks
- ☐ taskManager:project_delete /taskManager/<project_id>/project_delete/
taskManager.views.project_delete
- ☐ taskManager:project_details /taskManager/<project_id>/project_details/
taskManager.views.project_details
- ☐ taskManager:task_complete /taskManager/<project_id>/task_complete/<task_id>
taskManager.views.task_complete
- ☐ taskManager:task_create /taskManager/<project_id>/task_create/
taskManager.views.task_create
- ☐ taskManager:task_delete /taskManager/<project_id>/task_delete/<task_id>
taskManager.views.task_delete
- ☐ taskManager:task_edit /taskManager/<project_id>/task_edit/<task_id>
taskManager.views.task_edit

- ☐ taskManager:manage_projects /taskManager/manage_projects/ taskManager.views.manage_projects
- ☐ taskManager:project_create /taskManager/project_create/ taskManager.views.project_create
- ☐ taskManager:project_list /taskManager/project_list/ taskManager.views.project_list
- ☐ taskManager:task_list /taskManager/task_list/ taskManager.views.task_list

Low

- ☐ index / taskManager.views.index
- ☐ api-root /api/ rest_framework.routers.view
- ☐ api-root /api/<drf_format_suffix:format> rest_framework.routers.view
- ☐ profile-list /api/userprofiles/ taskManager.serializers.UserProfileViewSet
- ☐ profile-detail /api/userprofiles// taskManager.serializers.UserProfileViewSet
- ☐ profile-detail /api/userprofiles/. taskManager.serializers.UserProfileViewSet
- ☐ profile-list /api/userprofiles./ taskManager.serializers.UserProfileViewSet
- ☐ user-list /api/users/ taskManager.serializers.UserViewSet
- ☐ user-detail /api/users// taskManager.serializers.UserViewSet
- ☐ user-detail /api/users/. taskManager.serializers.UserViewSet
- ☐ user-list /api/users./ taskManager.serializers.UserViewSet
- ☐ redoc /redoc/ drf_spectacular.views.view
- ☐ schema /schema/ drf_spectacular.views.view
- ☐ /static/ django.views.static.serve
- ☐ swagger-ui /swagger-ui/ drf_spectacular.views.view
- ☐ taskManager:index /taskManager/ taskManager.views.index
- ☐ taskManager:dashboard /taskManager/dashboard/ taskManager.views.dashboard
- ☐ taskManager:settings /taskManager/settings/ taskManager.views.tm_settings

Mapping / Authorization Decorators

- ☐ @login_required - requires `request.user.is_authenticated`; redirects to login if not. Used on nearly all `taskManager` and `chatbot` views.
- ☐ @user_passes_test(can_create_project) - checks `user.has_perm('taskManager.create_project')`. Used on `project_create`.
- ☐ @user_passes_test(can_edit_project) - checks `user.has_perm('taskManager.edit_project')`. Used on `project_edit`.
- ☐ @user_passes_test(can_delete_project) - checks `user.has_perm('taskManager.delete_project')`. Used on `project_delete`.
- ☐ @user_passes_test(lambda u: u.is_superuser) - restricts to superusers. Used on `view_all_users`.
- ☐ @csrf_exempt - disables CSRF protection for the view (also globally disabled via middleware). Used on `profile_by_id`, `reset_password`, `forgot_password`, `change_password`, `ping`.
- ☐ @require_POST - restricts view to POST requests only (method guard, not authz). Used on `chatbot` views `chat_send`, `session_new`, `session_delete`.

Mapping / Files

- ☐ taskManager/settings.py
 - ☐ **!!!NOT – VULNERABLE!!** CSRF Middleware appears to be disabled globally